

TP 2 : Reconnaissance Automatique de la Langue d'un Texte à l'aide de Modèles de Langue Statistiques

Christophe SERVAN, PhD

July 2022

1 Outils disponibles

- L'outil SRI-LM : permet la création et utilisation de modèles de langue à n-grammes pour représenter et traiter des textes
 - Utilisez le script disponible [ici](#)
 - Pour les MacOS, editez le fichier et changez utilisez cette ligne de commande `make World MACHINE_TYPE=i686-m64` par `make World MACHINE_TYPE=macosx-m64` .
- L'outil fastText : permet la création et utilisation de représentations de mots et d'apprendre des modèles de classification (installé lors du précédent TP)
- Deux textes de taille équivalente en deux langues différentes
 - Utiliser les [fichiers fournis](#)
 - Tokenisez les fichiers au besoin.

2 Travail à réaliser

2.1 Partie 1 :

Exercice 1 : Couper chaque fichier de chaque langue en deux parties contenant un nombre de lignes égal (par exemple, on coupe le fichier `fr.txt` en un fichier `train.fr` et un fichier `test.fr`)

- **Q1 :** Pourquoi doit-on utiliser des textes différents pour l'apprentissage et le test ?

Rappel du TP 1 : La commande linux `wc` donne le nombre de lignes d'un fichier texte ; la commande `head` (ou `tail`) permet de récupérer les n premières (ou dernières) lignes d'un fichier.

Exercice 2 : Construire un modèle de langue à trigrammes pour chaque fichier d'apprentissage créé précédemment en utilisant les commandes suivantes :

```
ngram-count -order 3 -text train.fr -write fr3g.bo -write-vocab fr.voc
ngram-count -read fr3g.bo -lm fr3g.BO -gt2min 1 -gt3min 2
```

- **Q1 :** Observez les fichiers `.voc` , `.bo` et `.BO` ; que contiennent ils ?
- **Q2 :** À votre avis, à quoi correspondent les symboles `<s>` et `</s>` ?
- **Q3 :** À quoi servent les options `-gt2min` et `-gt3min` ? Dans quels cas elles sont utiles/nécessaires ?
- **Q4 :** Quel est le trigramme le plus fréquent dans chacune des langues ?

Remarque : La première colonne du fichier `.B0` est homogène au logarithme d' une probabilité.

Aide : La commande unix `sort -n` permet de trier un fichier en fonction d' un de ses champs présent sous forme numérique.

Exercice 3 : Calculer la perplexité de chaque texte `test.LANG` par rapport à chaque modèle `LANG3g.B0`. Exemple d' utilisation de la commande `ngram` :

```
ngram -lm fr3g.B0 -ppl test.en
```

- **Q1** : Présenter les résultats sous forme de matrice.
- **Q2** : Comment peut-on construire un système de reconnaissance de la langue d' un texte en utilisant un modèle de langue stochastique ?

Exercice 4 : Refaire le meme système avec des modèles n-grammes de CARACTÈRES.

- **Q1** : Quelles sont les perplexités pour ce modèle de caractères ? Quels pourraient être les avantages/inconvénients du modèle de caractères par rapport au modèle de mots ?
- **Q2** : Refaire le modèle en remplaçant l' espace par un symbole quelconque (p. ex., `<SPACE>`). Quelles sont les perplexités pour ce modèle ?

Aide : Un modèle peut être considéré plus performant qu' un autre si, pour un texte de la même langue, la perplexité est inférieure et, pour un texte dans une autre langue, la perplexité est supérieure.

Exercice 5 : Étendre la matrice de perplexités pour toutes les autres langues .

- **Q1** : Vérifier quelles sont les paires de langues les plus faciles/difficiles à distinguer les unes des autres ?

2.2 Partie 2 :

Exercice 1 : Identification de langue avec `fastText`.

- **Q1** : Refaire la matrice de confusion avec un modèle appris avec `fasttext` en utilisant tous les corpus d' apprentissage.
- **Q2** : Quels sont les avantages que vous pouvez identifier à l' utilisation de `fasttext` par rapport aux modèles de langue ?

Aide : Il vous faudra apprendre un modèle en mode `supervised` afin de réussir cette exercice. Devant chacune des phrase il vous faudra rajouter un label sous la forme `__label__XX` où `XX` sera remplacé par le code de langue du corpus d' apprentissage (`fr` pour français, `en` pour anglais, etc.).